

**Федеральное государственное бюджетное образовательное
учреждение высшего образования
«Херсонский государственный педагогический университет»**

Институт информационных технологий

Кафедра программной инженерии и информационных технологий

ПРАКТИЧЕСКАЯ РАБОТА

по дисциплине:

«Современные операционные системы и базы данных»

на тему:

«Оболочка командной строки PowerShell»

Выполнил:

Студент гр.: 12-25РПм

Трифонов Д.С.

Проверил

Преподаватель:

Алешов В.В.

Оценка: _____

Дата: 18.02.2026

Херсон, 2026

ПРАКТИЧЕСКАЯ РАБОТА

Только файлы больше 10000 байт в каталоге Windows

```

Администратор: Windows PowerShell
PS C:\Users\root> Get-ChildItem C:\Windows -File | Where-Object { $_.Length -gt 10000 }

Каталог: C:\Windows

Mode                LastWriteTime         Length Name
-----
-a----          11.11.2020         1:49         79360 bfsvc.exe
-a----          11.01.2026        11:00         66100 DirectX.log
-a----          15.09.2018        10:28         35353 EnterpriseS.xml
-a----          11.11.2020         1:49       4443888 explorer.exe
-a----          11.11.2020         1:52       1071616 HelpPane.exe
-a----          15.09.2018        10:29         18432 hh.exe
-a----          15.09.2018        10:28         43131 mib.bin
-a----          11.11.2020         1:50       254464  notepad.exe
-a----          17.02.2026        18:57         58272 PFR0.log
-a----          05.12.2025        18:18       782680  py.exe
-a----          05.12.2025        18:18         53592 pyshellext.amd64.dll
-a----          05.12.2025        18:18         781144  pyw.exe
-a----          11.11.2020         1:52       358400  regedit.exe
-a----          08.02.2026        22:13         52878 setupact.log
-a----          11.11.2020         1:49       165376  splwow64.exe
-a----          15.09.2018        10:29         64512 twain_32.dll
-a----          15.09.2018        10:29         11776 winhlp32.exe
-a----          15.09.2018        19:43       316640  WMSysPr9.prx
-a----          15.09.2018        10:29         11264 write.exe

PS C:\Users\root>
  
```

Вывести в текстовый файл список свойств процесса возвращаемый командой Get-process и на экран их общее количество

```

Администратор: Windows PowerShell
Mode                LastWriteTime         Length Name
-----
-a----          11.11.2020         1:49         79360 bfsvc.exe
-a----          11.01.2026        11:00         66100 DirectX.log
-a----          15.09.2018        10:28         35353 EnterpriseS.xml
-a----          11.11.2020         1:49       4443888 explorer.exe
-a----          11.11.2020         1:52       1071616 HelpPane.exe
-a----          15.09.2018        10:29         18432 hh.exe
-a----          15.09.2018        10:28         43131 mib.bin
-a----          11.11.2020         1:50       254464  notepad.exe
-a----          17.02.2026        18:57         58272 PFR0.log
-a----          05.12.2025        18:18       782680  py.exe
-a----          05.12.2025        18:18         53592 pyshellext.amd64.dll
-a----          05.12.2025        18:18         781144  pyw.exe
-a----          11.11.2020         1:52       358400  regedit.exe
-a----          08.02.2026        22:13         52878 setupact.log
-a----          11.11.2020         1:49       165376  splwow64.exe
-a----          15.09.2018        10:29         64512 twain_32.dll
-a----          15.09.2018        10:29         11776 winhlp32.exe
-a----          15.09.2018        19:43       316640  WMSysPr9.prx
-a----          15.09.2018        10:29         11264 write.exe

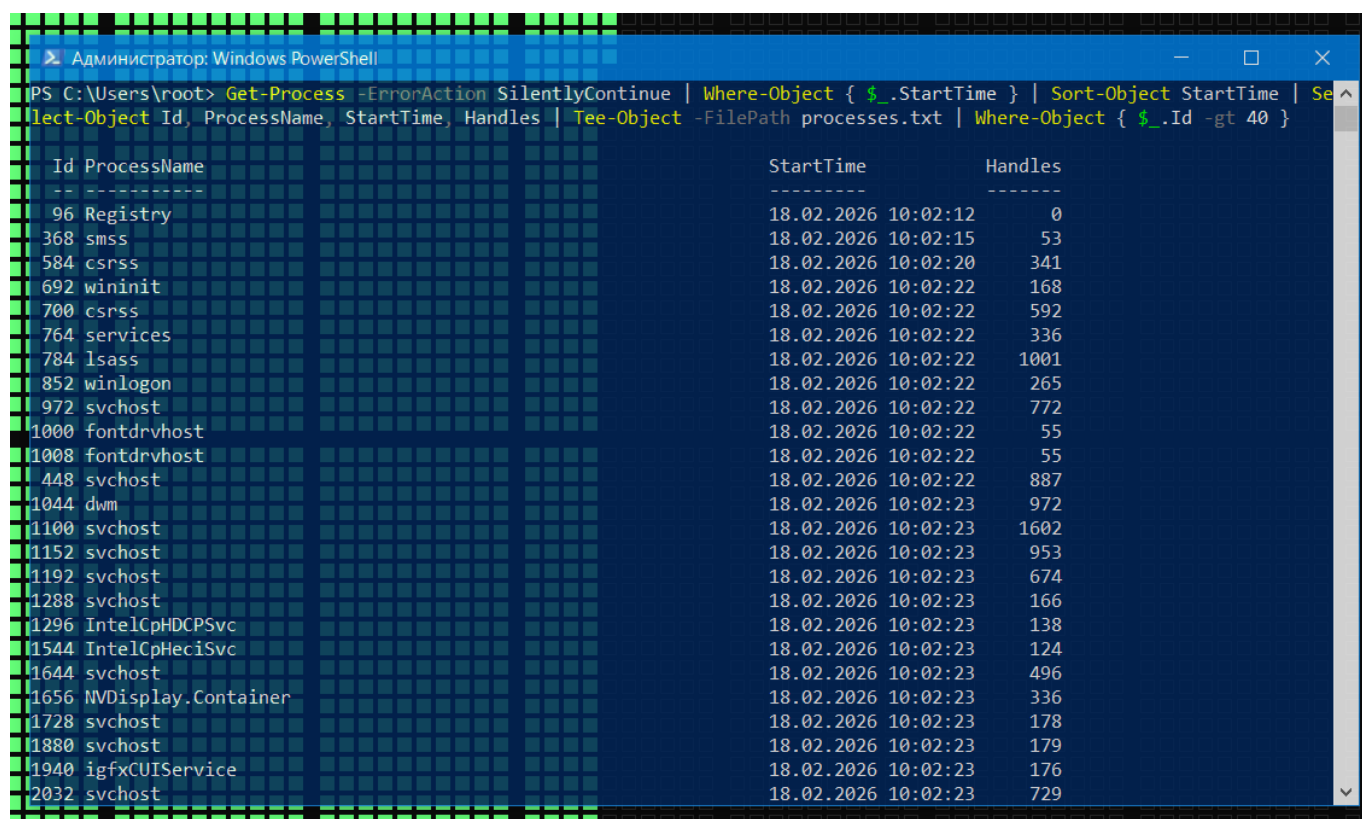
PS C:\Users\root> $properties = (Get-Process | Get-Member -MemberType Property).Name
PS C:\Users\root> $properties | Out-File process_properties.txt
PS C:\Users\root> $properties.Count
52
PS C:\Users\root>
  
```

process_properties.txt

```

Файл  Правка  Формат  Вид  Справка
BasePriority
Container
EnableRaisingEvents
ExitCode
ExitTime
Handle
HandleCount
HasExited
Id
MachineName
MainModule
MainWindowHandle
MainWindowTitle
MaxWorkingSet
MinWorkingSet
Modules
ManagedSystemMemory
  
```

Создание текстового файла со списком выполняемых процессов (Id, имя процесса, время старта, Handles), отсортированных по времени старта, и вывод на экран процессов с Id > 40



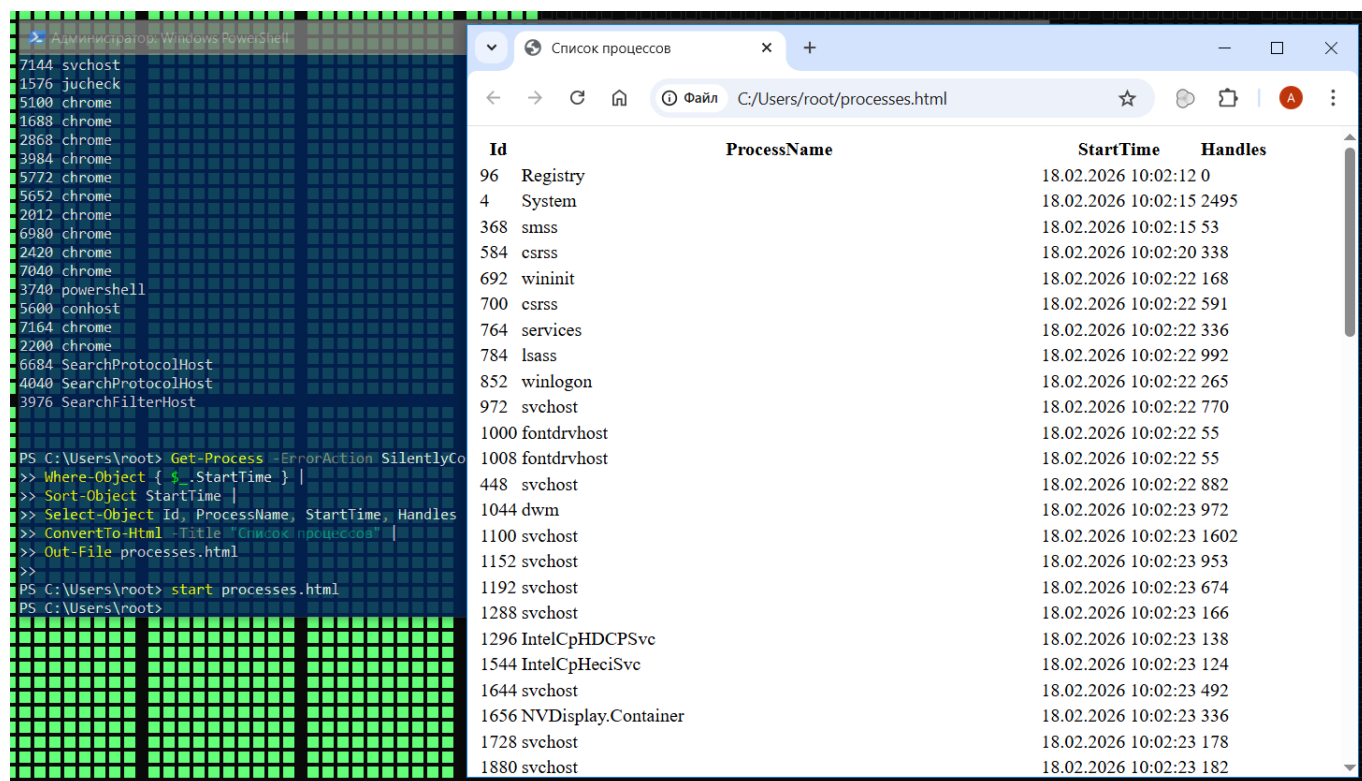
```

PS C:\Users\root> Get-Process -ErrorAction SilentlyContinue | Where-Object { $_.StartTime } | Sort-Object StartTime | Select-Object Id, ProcessName, StartTime, Handles | Tee-Object -FilePath processes.txt | Where-Object { $_.Id -gt 40 }

```

Id	ProcessName	StartTime	Handles
96	Registry	18.02.2026 10:02:12	0
368	smss	18.02.2026 10:02:15	53
584	csrss	18.02.2026 10:02:20	341
692	wininit	18.02.2026 10:02:22	168
700	csrss	18.02.2026 10:02:22	592
764	services	18.02.2026 10:02:22	336
784	lsass	18.02.2026 10:02:22	1001
852	winlogon	18.02.2026 10:02:22	265
972	svchost	18.02.2026 10:02:22	772
1000	fontdrvhost	18.02.2026 10:02:22	55
1008	fontdrvhost	18.02.2026 10:02:22	55
448	svchost	18.02.2026 10:02:22	887
1044	dwm	18.02.2026 10:02:23	972
1100	svchost	18.02.2026 10:02:23	1602
1152	svchost	18.02.2026 10:02:23	953
1192	svchost	18.02.2026 10:02:23	674
1288	svchost	18.02.2026 10:02:23	166
1296	IntelCpHDCPSvc	18.02.2026 10:02:23	138
1544	IntelCpHeciSvc	18.02.2026 10:02:23	124
1644	svchost	18.02.2026 10:02:23	496
1656	NVDisplay.Container	18.02.2026 10:02:23	336
1728	svchost	18.02.2026 10:02:23	178
1880	svchost	18.02.2026 10:02:23	179
1940	igfxCUIService	18.02.2026 10:02:23	176
2032	svchost	18.02.2026 10:02:23	729

Создание HTML-файла со списком выполняемых процессов (Id, имя процесса, время старта, Handles), упорядоченных по возрастанию времени старта



```

PS C:\Users\root> Get-Process -ErrorAction SilentlyContinue
>> Where-Object { $_.StartTime } |
>> Sort-Object StartTime |
>> Select-Object Id, ProcessName, StartTime, Handles
>> ConvertTo-Html -Title "Список процессов" |
>> Out-File processes.html
>>
PS C:\Users\root> start processes.html
PS C:\Users\root>

```

Id	ProcessName	StartTime	Handles
96	Registry	18.02.2026 10:02:12	0
4	System	18.02.2026 10:02:15	2495
368	smss	18.02.2026 10:02:15	53
584	csrss	18.02.2026 10:02:20	338
692	wininit	18.02.2026 10:02:22	168
700	csrss	18.02.2026 10:02:22	591
764	services	18.02.2026 10:02:22	336
784	lsass	18.02.2026 10:02:22	992
852	winlogon	18.02.2026 10:02:22	265
972	svchost	18.02.2026 10:02:22	770
1000	fontdrvhost	18.02.2026 10:02:22	55
1008	fontdrvhost	18.02.2026 10:02:22	55
448	svchost	18.02.2026 10:02:22	882
1044	dwm	18.02.2026 10:02:23	972
1100	svchost	18.02.2026 10:02:23	1602
1152	svchost	18.02.2026 10:02:23	953
1192	svchost	18.02.2026 10:02:23	674
1288	svchost	18.02.2026 10:02:23	166
1296	IntelCpHDCPSvc	18.02.2026 10:02:23	138
1544	IntelCpHeciSvc	18.02.2026 10:02:23	124
1644	svchost	18.02.2026 10:02:23	492
1656	NVDisplay.Container	18.02.2026 10:02:23	336
1728	svchost	18.02.2026 10:02:23	178
1880	svchost	18.02.2026 10:02:23	182

Найти суммарный объем всех графических файлов bmp и jpg в каталоге windows и всех подкаталогах

```

Администратор: Windows PowerShell

1688 chrome      18.02.2026 13:38:56 182
2868 chrome      18.02.2026 13:38:56 922
3984 chrome      18.02.2026 13:38:56 346
5772 chrome      18.02.2026 13:38:57 206
5652 chrome      18.02.2026 13:38:57 226
2012 chrome      18.02.2026 13:38:57 217
6980 chrome      18.02.2026 13:38:57 197
2420 chrome      18.02.2026 13:38:58 249
7040 chrome      18.02.2026 13:38:58 220
3740 powershell  18.02.2026 13:40:15 716
5600 conhost     18.02.2026 13:40:15 255
7164 chrome      18.02.2026 13:40:59 395
2200 chrome      18.02.2026 13:41:01 168
6684 SearchProtocolHost 18.02.2026 13:42:56 298
4040 SearchProtocolHost 18.02.2026 13:44:47 374
3976 SearchFilterHost 18.02.2026 13:47:44 149

PS C:\Users\root> Get-Process -ErrorAction SilentlyContinue |
>> Where-Object { $_.StartTime } |
>> Sort-Object StartTime |
>> Select-Object Id, ProcessName, StartTime, Handles |
>> ConvertTo-Html -Title "Список процессов" |
>> Out-File processes.html
>>
PS C:\Users\root> start processes.html
PS C:\Users\root> (Get-ChildItem C:\Windows -Recurse -Include *.bmp, *.jpg -File -ErrorAction SilentlyContinue | Measure
-Object -Property Length -Sum).Sum
24014434
PS C:\Users\root>

```

Вывести на экран сведения о ЦП компьютера

```

Администратор: Windows PowerShell

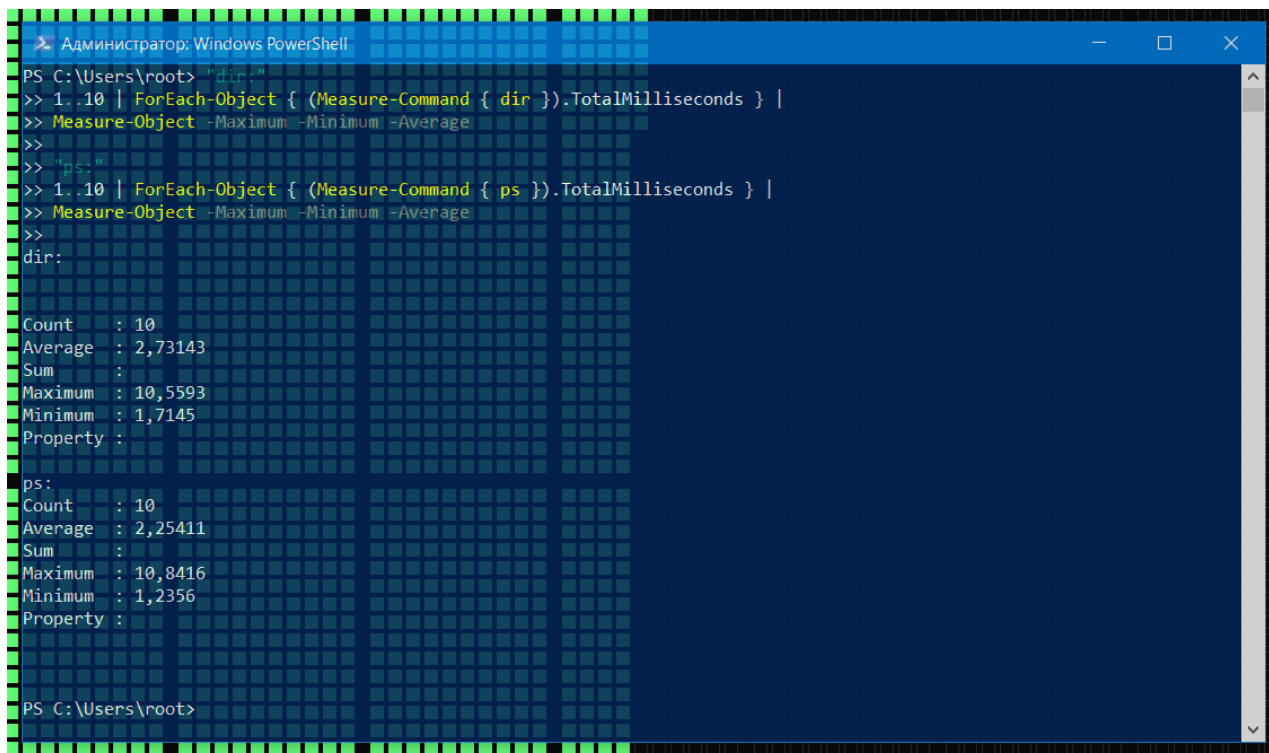
PS C:\Users\root> Get-CimInstance Win32_Processor | Select-Object Name, Manufacturer, NumberOfCores, NumberOfLogicalProc
essors, MaxClockSpeed

Name                : Intel(R) Core(TM) i3-6006U CPU @ 2.00GHz
Manufacturer        : GenuineIntel
NumberOfCores        : 2
NumberOfLogicalProcessors : 4
MaxClockSpeed        : 1992

PS C:\Users\root>

```

Найти максимальное, минимальное и среднее значение времени выполнения командлетов dir и ps



```

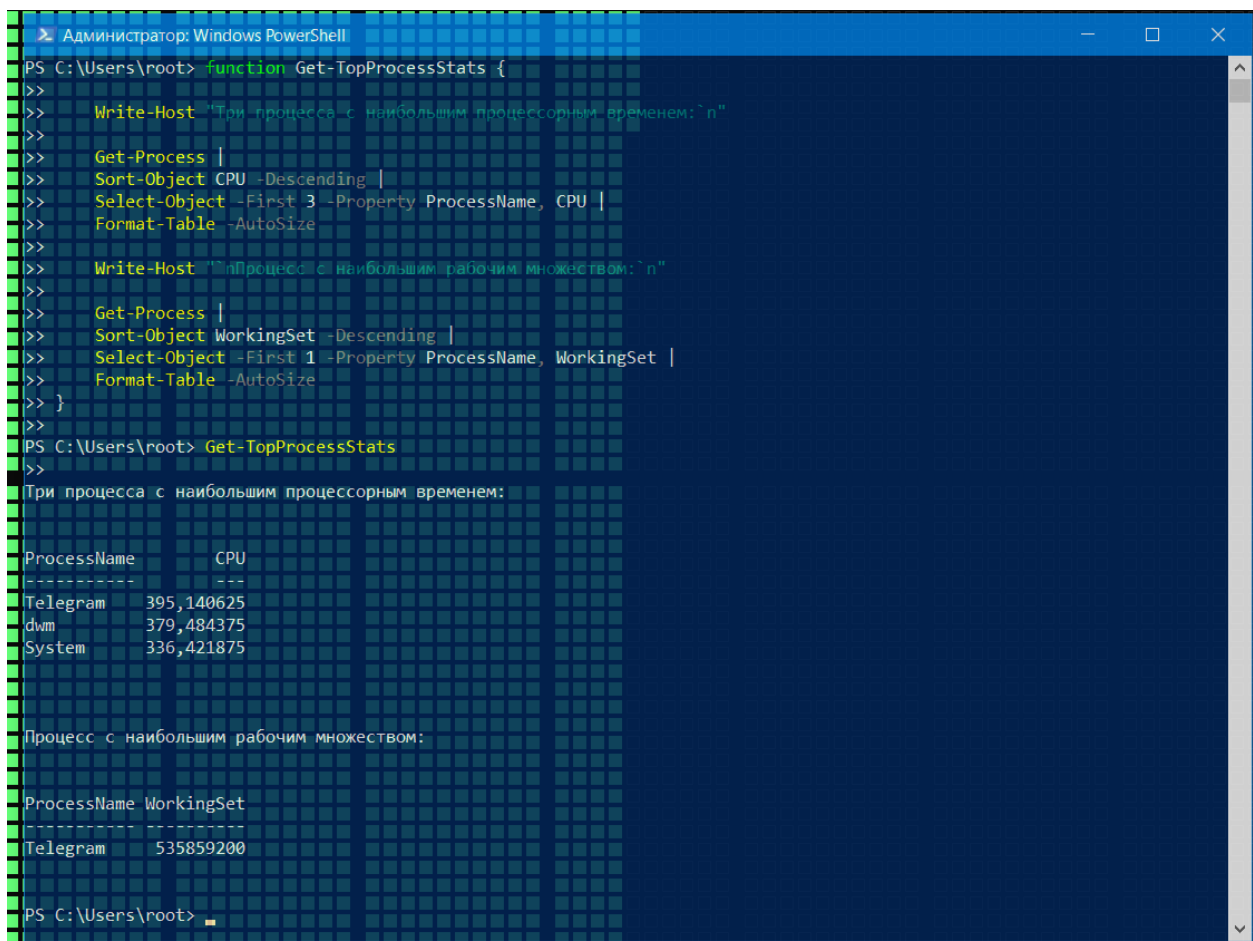
Администратор: Windows PowerShell
PS C:\Users\root> "dir"
>> 1..10 | ForEach-Object { (Measure-Command { dir }).TotalMilliseconds } |
>> Measure-Object -Maximum -Minimum -Average
>>
>> "ps"
>> 1..10 | ForEach-Object { (Measure-Command { ps }).TotalMilliseconds } |
>> Measure-Object -Maximum -Minimum -Average
>>
dir:
Count      : 10
Average    : 2,73143
Sum        :
Maximum    : 10,5593
Minimum    : 1,7145
Property   :

ps:
Count      : 10
Average    : 2,25411
Sum        :
Maximum    : 10,8416
Minimum    : 1,2356
Property   :

PS C:\Users\root>

```

Разработать командлет для нахождения среди выполняющихся процессов имен трех процессов, использовавших более всего процессорного времени нахождения среди выполняющихся процессов имени процесса с наибольшим размером рабочего множества страниц



```

Администратор: Windows PowerShell
PS C:\Users\root> function Get-TopProcessStats {
>>
>>     Write-Host "Три процесса с наибольшим процессорным временем: `n"
>>
>>     Get-Process |
>>     Sort-Object CPU -Descending |
>>     Select-Object -First 3 -Property ProcessName, CPU |
>>     Format-Table -AutoSize
>>
>>     Write-Host "`nПроцесс с наибольшим рабочим множеством: `n"
>>
>>     Get-Process |
>>     Sort-Object WorkingSet -Descending |
>>     Select-Object -First 1 -Property ProcessName, WorkingSet |
>>     Format-Table -AutoSize
>> }
>>
PS C:\Users\root> Get-TopProcessStats
>>
Три процесса с наибольшим процессорным временем:

ProcessName      CPU
-----
Telegram         395,140625
dwm              379,484375
System          336,421875

Процесс с наибольшим рабочим множеством:

ProcessName WorkingSet
-----
Telegram    535859200

PS C:\Users\root>

```

Ответы на контрольные вопросы

1. Типы команд PowerShell (PS)

- Командлеты (cmdlet) — базовые встроенные команды PowerShell, реализованные как .NET-классы, работают с объектами, а не с текстом.
- Функции — команды, определённые в скриптах или профиле пользователя, могут использовать ту же схему Verb-Noun и работать как командлеты.
- Скрипты (.ps1) — файлы с последовательностью команд и логикой, выполняемые как единая команда.
- Приложения/консольные утилиты (native executables) — внешние программы, запускаемые из PowerShell (например, ping.exe, robocopy.exe).
- Алиасы — альтернативные короткие имена для командлетов, функций или скриптов (например, ls для Get-ChildItem).
- Модули — наборы командлетов, функций, алиасов и др., которые подключаются командой Import-Module.

2. Имена и структура командлетов

- Имя командлета всегда имеет форму Verb-Noun (глагол-существительное), например Get-Process, Set-Item, Start-Service.
- Глагол описывает действие (Get — получить данные, Set — изменить, New — создать, Remove — удалить, Start — запустить и т.д.).
- Существительное обозначает тип объекта, над которым выполняется действие (Process, Item, Service, User и т.п.), используется в единственном числе и без глаголов.
- Для глаголов используется утверждённый перечень “approved verbs”, что обеспечивает единообразие и предсказуемость имен.
- Командлеты имеют стандартную структуру параметров, поддерживают именованные и позиционные параметры, конвейерный ввод и общие параметры (например, -Verbose, -ErrorAction).

3. Псевдонимы команд

- Алиас (alias) — это альтернативное имя для существующей команды (командлета, функции, скрипта или приложения), используемое для сокращения и удобства.
- Примеры: ls, dir, gci — алиасы для Get-ChildItem; gc — для Get-Content; rm — для Remove-Item; cd — для Set-Location.
- Просмотр алиасов: Get-Alias (gal) выводит список всех текущих псевдонимов и связанных с ними команд.
- Создание и изменение алиасов: New-Alias и Set-Alias позволяют назначать или переопределять псевдонимы в текущем сеансе.
- Алиасы предназначены для интерактивной работы; в производственных скриптах рекомендуется использовать полные имена командлетов для ясности.

4. Просмотр структуры объектов

- PowerShell работает с объектами .NET, поэтому каждая команда возвращает объекты с набором свойств и методов.
- Для исследования типа объекта и его членов используется командлет `Get-Member` (`gm`), который показывает свойства, методы, события и базовые типы.
- Пример: `Get-Process | Get-Member` — выводит структуру объектов процесса (имена и типы свойств, доступные методы и т.д.).
- Дополнительно `Out-GridView`, `Format-List` и `Format-Table` помогают визуально просмотреть отдельные поля и значения свойств объектов.

5. Фильтрация объектов в конвейере. Блок сценария

- Конвейер PowerShell передаёт между командами не текст, а объекты, что позволяет фильтровать их по свойствам на любом этапе обработки.
- Основной инструмент фильтрации — командлет `Where-Object` (короткое имя `where`), который выбирает объекты с нужными значениями свойств.
- `Where-Object` использует блок сценария (script block) в фигурных скобках `{ }`, внутри которого задаётся логическое условие с обращением к текущему объекту через `$_`.
- Пример скриптового блока: `Get-Service | Where-Object { $_.Status -eq 'Running' }` — в конвейер дальше попадут только службы со статусом `Running`.
- Начиная с PowerShell 3.0 доступен “упрощённый синтаксис”: `Get-Process | Where-Object PriorityClass -EQ 'Normal'`, где свойство и оператор задаются параметрами командлета.